

Database Management Systems

CS 5200

- Announcements:
 - March 20th Lecture will be Asynchronous.
 - Religious Holiday for some of us.
 - Midterm Grades are published.
 - Makeup?

Plan for Today

- Module 9: Chapter 10 - Database Programming (75 minutes)
- Discussion (15 minutes)
 - Class Activity: Practice Exercises
- Break (20 minutes)
- Module 9: Chapter 14 - Normalization (75 minutes)
- Discussion (10 minutes)
 - Class Activity: Practice Exercises
 - Review Deliverables (5 minutes)

Show and Drop

- Use **show procedure status** to display the list of stored procedures you have created

Db	Name	Type	Definer	Modified	Created	Security_type
silicon	updateSalary	PROCEDURE	root@localhost	2022-10-07 22:00:45	2022-10-07 22:00:45	DEFINER

- Use **drop procedure** to remove a stored procedure

```
mysql> drop procedure updateSalary;  
Query OK, 0 rows affected (0.00 sec)
```

- MySQL also supports cursors in stored procedures.
 - A cursor is used to iterate through a set of rows returned by a query so that we can process each individual row.
- To learn more about stored procedures, go to:
<http://www.mysqltutorial.org/mysql-stored-procedure-tutorial.aspx>

Conditions and Handlers

- Condition: like an exception
 - You can declare your own conditions
 - Beyond the scope of the course
- Handler: like the catch block in a try/catch construct
 - MySQL's own handlers are enough for the purposes of this course.

Chapter 10 Outline

- Database Programming: Techniques and Issues
- Embedded SQL, Dynamic SQL, and SQLJ
- Database Programming with Function Calls: SQL/CLI and JDBC
- Database Stored Procedures and SQL/PSM
- Comparing the Three Approaches

Database Programming

- Objective:

- To access a database from an application program (as opposed to interactive interfaces)

- Why?

- An interactive interface is convenient but not sufficient
- A majority of database operations are made through application programs (increasingly through web applications)

SQL Programming Techniques

- **Database applications**
 - **Host language**
 - Python, Java, C/C++/C#, COBOL, or some other programming language
 - **Data sublanguage**
 - SQL
- **SQL standards**
 - Continually evolving
 - Each DBMS vendor may have some variations from standard

Database Programming: Techniques and Issues

- **Interactive interface**
 - SQL commands typed directly into a monitor
- **Execute file of commands**
 - *@<filename>*
- **Application programs or database applications**
 - Used as canned transactions by the end users access a database
 - May have **Web interface**

Approaches to Database Programming

- **Embedding** database commands in a general-purpose programming language
 - Database statements identified by a special prefix
 - **Precompiler** or **preprocessor** scans the source program code
 - Identify database statements and extract them for processing by the DBMS
 - Called **embedded SQL**
- Using a library of database functions
 - **Library of functions** available to the host programming language
 - **Application programming interface (API)**
- Designing a brand-new language
 - **Database programming language** designed from scratch
- First two approaches are more common

Impedance Mismatch

- Differences between database model and programming language model
 - e.g. Data types in programming language may differ from types in relational model.
- **Solution: Binding** for each host programming language
 - Specifies for each attribute type the compatible programming language types
- Cursor or iterator variable
 - Loop over the tuples in a query result
- Less of a problem when we use a special language design to be used with a database model:
 - e.g. Oracle's PL/SQL.

Typical Sequence of Interaction in Database Programming

- Open a connection to database server
- Interact with database by submitting queries, updates, and other database commands
- Terminate or close connection to database

Embedded SQL, Dynamic SQL and SQLJ

- **Embedded SQL**
 - C language
- **SQLJ**
 - Java language
- Programming language called **host language**

Embedded SQL: Retrieving Single Tuples

- EXEC SQL
 - Prefix
 - **Preprocessor** separates embedded SQL statements from host language code
 - Terminated by a matching END-EXEC
 - Or by a semicolon (;)
- **Shared variables**
 - Used in both the C program and the embedded SQL statements
 - Prefixed by a colon (:) in SQL statement

Figure 10.1

C program variables used in the embedded SQL examples E1 and E2.

```
0)  int  loop  ;
1)  EXEC SQL BEGIN DECLARE SECTION ;
2)  varchar dname [16], fname [16], lname [16], address [31] ;
3)  char  ssn [10], bdate [11], sex [2], minit [2] ;
4)  float salary, raise ;
5)  int  dno, dnumber ;
6)  int  SQLCODE ; char  SQLSTATE [6] ;
7)  EXEC SQL END DECLARE SECTION ;
```

- Variables declared between SQL BEGIN and SQL END are SQL variables.
 - **SQLCODE** and **SQLSTATE** are SQL Communication variables.
 - Used to communicate errors and exceptions
 - **SQLCODE** variable
 - 0 = statement executed successfully
 - 100 = no more data available in query result
 - < 0 = indicates some error has occurred
 - **SQLSTATE**
 - String of five characters
 - '00000' = no error or exception
 - Other values indicate various errors or exceptions
 - For example, '02000' indicates 'no more data' when using SQLSTATE
- Loop is a C variable

Retrieving Single Tuples with Embedded SQL (cont'd.)

- Connecting to the database

```
CONNECT TO <server name>AS <connection name>  
AUTHORIZATION <user account name and password> ;
```

- Change connection

```
SET CONNECTION <connection name> ;
```

- Terminate connection

```
DISCONNECT <connection name> ;
```

Figure 10.2

Program segment E1, a C program segment with embedded SQL.

```
//Program Segment E1:
0) loop = 1 ;
1) while (loop) {
2)     prompt("Enter a Social Security Number: ", ssn) ;
3)     EXEC SQL
4)         SELECT Fname, Minit, Lname, Address, Salary
5)         INTO :fname, :minit, :lname, :address, :salary
6)         FROM EMPLOYEE WHERE Ssn = :ssn ;
7)     if (SQLCODE == 0) printf(fname, minit, lname, address, salary)
8)     else printf("Social Security Number does not exist: ", ssn) ;
9)     prompt("More Social Security Numbers (enter 1 for Yes, 0 for No): ", loop) ;
10) }
```

■ INTO

- Error if multiple records are returned.

■ Line 7: SQLCODE

Retrieving Multiple Tuples with Embedded SQL Using Cursors

- **Cursor**
 - Points to a single tuple (row) from result of query
- **OPEN CURSOR** command
 - Fetches query result and sets cursor to a position before first row in result
 - Becomes current row for cursor
- **FETCH** commands
 - Moves cursor to next row in result of query

Figure 10.3 Program segment E2,
a C program segment that uses cursors with embedded SQL for update purposes.

```
//Program Segment E2:
0) prompt("Enter the Department Name: ", dname) ;
1) EXEC SQL
2)     SELECT Dnumber INTO :dnumber
3)     FROM DEPARTMENT WHERE Dname = :dname ;
4) EXEC SQL DECLARE EMP CURSOR FOR
5)     SELECT Ssn, Fname, Minit, Lname, Salary
6)     FROM EMPLOYEE WHERE Dno = :dnumber
7)     FOR UPDATE OF Salary ;
8) EXEC SQL OPEN EMP ;
9) EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit, :lname, :salary ;
10) while (SQLCODE == 0) {
11)     printf("Employee name is:", Fname, Minit, Lname) ;
12)     prompt("Enter the raise amount: ", raise) ;
13)     EXEC SQL
14)         UPDATE EMPLOYEE
15)         SET Salary = Salary + :raise
16)         WHERE CURRENT OF EMP ;
17)     EXEC SQL FETCH FROM EMP INTO :ssn, :fname, :minit, :lname, :salary ;
18) }
19) EXEC SQL CLOSE EMP ;
```

Retrieving Multiple Tuples with Embedded SQL Using Cursors (cont'd.)

```
DECLARE <cursor name> [ INSENSITIVE ] [ SCROLL ] CURSOR  
[ WITH HOLD ] FOR <query specification>  
[ ORDER BY <ordering specification> ]  
[ FOR READ ONLY | FOR UPDATE [ OF <attribute list> ] ] ;
```

■ FOR UPDATE OF

- List the names of any attributes that will be updated by the program
- Nothing listed means deleting a whole tuple

■ SCROLL position cursor to be used in other than sequential

- Fetch orientation
 - Added using value: NEXT, PRIOR, FIRST, LAST, ABSOLUTE *i*, and RELATIVE *i*

■ READ ONLY is the default

```
FETCH [ [ <fetch orientation> ] FROM ] <cursor name> INTO <fetch target list>;
```

Specifying Queries at Runtime Using Dynamic SQL

- **Dynamic SQL**
 - Execute different SQL queries or updates dynamically at runtime
- **Dynamic update**
 - Simple (next slide)
- **Dynamic query**
 - More involved since the programmer does not know what variables are returned by dynamic query.

Figure 10.4 Program segment E3, a C program segment that uses dynamic SQL for updating a table.

```
//Program Segment E3:
0) EXEC SQL BEGIN DECLARE SECTION ;
1) varchar sqlupdatestring [256] ;
2) EXEC SQL END DECLARE SECTION ;
   ...
3) prompt("Enter the Update Command: ", sqlupdatestring) ;
4) EXEC SQL PREPARE sqlcommand FROM :sqlupdatestring ;
5) EXEC SQL EXECUTE sqlcommand ;
   ...

EXEC SQL EXECUTE IMMEDIATE :sqlupdatestring ;
```

SQLJ:

Embedding SQL Commands in Java

- Standard adopted by several vendors for embedding SQL in Java
- Import several class libraries
- **Default context** is set for subsequent commands
- Uses **exceptions** for error handling
 - `SQLException` is used to return errors or exception conditions

Figure 10.5 Importing classes needed for including SQLJ in Java programs in Oracle, and establishing a connection and default context.

```
1) import java.sql.* ;
2) import java.io.* ;
3) import sqlj.runtime.* ;
4) import sqlj.runtime.ref.* ;
5) import oracle.sqlj.runtime.* ;
   ...
6) DefaultContext cntxt =
7) oracle.getConnection("<url name>", "<user name>", "<password>", true) ;
8) DefaultContext.setDefaultContext(cntxt) ;
   ...
```

Figure 10.6 Java program variables used in SQLJ examples J1 and J2.

- 1) `string dname, ssn , fname, fn, lname, ln,
bdate, address ;`
- 2) `char sex, minit, mi ;`
- 3) `double salary, sal ;`
- 4) `integer dno, dnumber ;`

Figure 10.7 Program segment J1, a Java program segment with SQLJ.

```
//Program Segment J1:
1) ssn = readEntry("Enter a Social Security Number: ") ;
2) try {
3)     #sql { SELECT Fname, Minit, Lname, Address, Salary
4)         INTO :fname, :minit, :lname, :address, :salary
5)         FROM EMPLOYEE WHERE Ssn = :ssn} ;
6) } catch (SQLException se) {
7)     System.out.println("Social Security Number does not exist: " + ssn) ;
8)     Return ;
9) }
10) System.out.println(fname + " " + minit + " " + lname + " " + address
    + " " + salary)
```

Retrieving Multiple Tuples in SQLJ Using Iterators

- **Iterator**

- Object associated with a collection (set or multiset) of records in a query result

- **Named iterator**

- Associated with a query result by listing attribute names and types in query result

- **Positional iterator**

- Lists only attribute types in query result

Figure 10.8 Program segment J2A, a Java program segment that uses a **named iterator** to print employee information in a particular department.

```
//Program Segment J2A:
0) dname = readEntry("Enter the Department Name: ") ;
1) try {
2)     #sql { SELECT Dnumber INTO :dnumber
3)         FROM DEPARTMENT WHERE Dname = :dname} ;
4) } catch (SQLException se) {
5)     System.out.println("Department does not exist: " + dname) ;
6)     Return ;
7) }
8) System.out.println("Employee information for Department: " + dname) ;
9) #sql iterator Emp(String ssn, String fname, String minit, String lname,
    double salary) ;
10) Emp e = null ;
11) #sql e = { SELECT ssn, fname, minit, lname, salary
12)     FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) while (e.next()) {
14)     System.out.println(e.ssn + " " + e.fname + " " + e.minit + " " +
        e.lname + " " + e.salary) ;
15) } ;
16) e.close() ;
```

Figure 10.9 Program segment J2B, a Java program segment that uses a **positional iterator** to print employee information in a particular department.

```
//Program Segment J2B:
0) dname = readEntry("Enter the Department Name: ") ;
1) try {
2)     #sql { SELECT Dnumber INTO :dnumber
3)         FROM DEPARTMENT WHERE Dname = :dname} ;
4) } catch (SQLException se) {
5)     System.out.println("Department does not exist: " + dname) ;
6)     Return ;
7) }
8) System.out.println("Employee information for Department: " + dname) ;
9) #sql iterator Emppos(String, String, String, String, double) ;
10) Emppos e = null ;
11) #sql e = { SELECT ssn, fname, minit, lname, salary
12)     FROM EMPLOYEE WHERE Dno = :dnumber} ;
13) #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
14) while (!e.endFetch()) {
15)     System.out.println(ssn + " " + fn + " " + mi + " " + ln + " " + sal) ;
16)     #sql { FETCH :e INTO :ssn, :fn, :mi, :ln, :sal} ;
17) } ;
18) e.close() ;
```

Database Programming with Function Calls: SQL/CLI & JDBC

- Use of function calls
 - **Dynamic** approach for database programming
- Library of functions
 - Also known as **application programming interface (API)**
 - Used to access database
- **SQL Call Level Interface (SQL/CLI)**
 - Part of SQL standard

SQL/CLI: Using C as the Host Language

- **Environment record**
 - Track one or more database connections
 - Set environment information
- **Connection record**
 - Keeps track of information needed for a particular database connection
- **Statement record**
 - Keeps track of the information needed for one SQL statement
- **Description record**
 - Keeps track of information about tuples or parameters
- **Handle to the record**
 - C pointer variable makes record accessible to program

```

//Program CLI1:
0) #include sqlcli.h ;
1) void printSal() {
2) SQLHSTMT stmt1 ;
3) SQLHDBC con1 ;
4) SQLHENV env1 ;
5) SQLRETURN ret1, ret2, ret3, ret4 ;
6) ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;
7) if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;
8) if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz",
    SQL_NTS) else exit ;
9) if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;
10) SQLPrepare(stmt1, "select Lname, Salary from EMPLOYEE where Ssn = ?",
    SQL_NTS) ;
11) prompt("Enter a Social Security Number: ", ssn) ;
12) SQLBindParameter(stmt1, 1, SQL_CHAR, &ssn, 9, &fetchlen1) ;
13) ret1 = SQLExecute(stmt1) ;
14) if (!ret1) {
15)     SQLBindCol(stmt1, 1, SQL_CHAR, &lname, 15, &fetchlen1) ;
16)     SQLBindCol(stmt1, 2, SQL_FLOAT, &salary, 4, &fetchlen2) ;
17)     ret2 = SQLFetch(stmt1) ;
18)     if (!ret2) printf(ssn, lname, salary)
19)         else printf("Social Security Number does not exist: ", ssn) ;
20) }
21) }

```



```

//Program Segment CLI2:
0) #include sqlcli.h ;
1) void printDepartmentEmps() {
2) SQLHSTMT stmt1 ;
3) SQLHDBC con1 ;
4) SQLHENV env1 ;
5) SQLRETURN ret1, ret2, ret3, ret4 ;
6) ret1 = SQLAllocHandle(SQL_HANDLE_ENV, SQL_NULL_HANDLE, &env1) ;
7) if (!ret1) ret2 = SQLAllocHandle(SQL_HANDLE_DBC, env1, &con1) else exit ;
8) if (!ret2) ret3 = SQLConnect(con1, "dbs", SQL_NTS, "js", SQL_NTS, "xyz",
    SQL_NTS) else exit ;
9) if (!ret3) ret4 = SQLAllocHandle(SQL_HANDLE_STMT, con1, &stmt1) else exit ;
10) SQLPrepare(stmt1, "select Lname, Salary from EMPLOYEE where Dno = ?",
    SQL_NTS) ;
11) prompt("Enter the Department Number: ", dno) ;
12) SQLBindParameter(stmt1, 1, SQL_INTEGER, &dno, 4, &fetchlen1) ;
13) ret1 = SQLExecute(stmt1) ;
14) if (!ret1) {
15)     SQLBindCol(stmt1, 1, SQL_CHAR, &lname, 15, &fetchlen1) ;
16)     SQLBindCol(stmt1, 2, SQL_FLOAT, &salary, 4, &fetchlen2) ;
17)     ret2 = SQLFetch(stmt1) ;
18)     while (!ret2) {
19)         printf(lname, salary) ;
20)         ret2 = SQLFetch(stmt1) ;
21)     }
22) }
23) }

```


JDBC:

SQL Function Calls for Java Programming

- JDBC
 - Java function libraries
- Single Java program can connect to several different databases
 - Called data sources accessed by the Java program
- `Class.forName("oracle.jdbc.driver.OracleDriver")`
 - Load a **JDBC driver** explicitly

JDBC:

SQL Function Calls for Java Programming

- **Connection object**
- **Statement object** has two subclasses:
 - PreparedStatement and CallableStatement
- **Question mark (?) symbol**
 - Represents a statement parameter
 - Determined at runtime
- **ResultSet object**
 - Holds results of query

```

//Program JDBC1:
0) import java.io.* ;
1) import java.sql.*
   ...
2) class getEmpInfo {
3)     public static void main (String args []) throws SQLException, IOException {
4)         try { Class.forName("oracle.jdbc.driver.OracleDriver")
5)             } catch (ClassNotFoundException x) {
6)                 System.out.println ("Driver could not be loaded") ;
7)             }
8)         String dbacct, passwrld, ssn, lname ;
9)         Double salary ;
10)        dbacct = readentry("Enter database account:") ;
11)        passwrld = readentry("Enter password:") ;
12)        Connection conn = DriverManager.getConnection
13)            ("jdbc:oracle:oci8:" + dbacct + "/" + passwrld) ;
14)        String stmt1 = "select Lname, Salary from EMPLOYEE where Ssn = ?" ;
15)        PreparedStatement p = conn.prepareStatement(stmt1) ;
16)        ssn = readentry("Enter a Social Security Number: ") ;
17)        p.clearParameters() ;
18)        p.setString(1, ssn) ;
19)        ResultSet r = p.executeQuery() ;
20)        while (r.next()) {
21)            lname = r.getString(1) ;
22)            salary = r.getDouble(2) ;
23)            system.out.println(lname + salary) ;
24)        } }
25) }

```

Database Stored Procedures and SQL/PSM

- **Stored procedures**
 - Program modules stored by the DBMS at the database server
 - Can be functions or procedures
- **SQL/PSM (SQL/Persistent Stored Modules)**
 - The standardized language framework
 - Extensions to SQL
 - Include general-purpose programming constructs in SQL
- **Useful:**
 - When database program is needed by several applications
 - To reduce data transfer and communication cost between client and server in certain situations
 - To enhance modeling power provided by views

Database Stored Procedures and Functions (cont'd.)

- Declaring stored procedures:

```
CREATE PROCEDURE <procedure name> (<parameters>)  
<local declarations>  
<procedure body> ;
```

- Declaring a function, a return type is necessary, so the declaration form is

```
CREATE FUNCTION <function name> (<parameters>)  
RETURNS <return type>  
<local declarations>  
<function body> ;
```

Database Stored Procedures and Functions (cont'd.)

- Each parameter has parameter type
 - **Parameter type:** one of the SQL data types
 - **Parameter mode:** IN, OUT, or INOUT
- Calling a stored procedure:
`CALL <procedure or function name> (<argument list>) ;`
- You can declare variables in stored procedures
- Can have any number of parameters.
- varchar declarations for the parameters have to specify the maximum length.
- flow control (IF-THEN-ELSE) or loops (WHILE and REPEAT) statements can be used in a procedure or function

SQL/PSM: Extending SQL for Specifying Persistent Stored Modules

- Conditional branching statement:

```
IF <condition> THEN <statement list>  
ELSEIF <condition> THEN <statement list>  
...  
ELSEIF <condition> THEN <statement list>  
ELSE <statement list>  
END IF ;
```

Figure 10.14 Declaring a function in SQL/PSM.

```
//Function PSM1:  
0) CREATE FUNCTION Dept_size(IN deptno INTEGER)  
1) RETURNS VARCHAR [7]  
2) DECLARE No_of_ems INTEGER ;  
3) SELECT COUNT(*) INTO No_of_ems  
4) FROM EMPLOYEE WHERE Dno = deptno ;  
5) IF No_of_ems > 100 THEN RETURN "HUGE"  
6) ELSEIF No_of_ems > 25 THEN RETURN "LARGE"  
7) ELSEIF No_of_ems > 10 THEN RETURN "MEDIUM"  
8) ELSE RETURN "SMALL"  
9) END IF ;
```


Case Statement

- Two different syntaxes:

```
CASE <expression>
  WHEN <value> then
    <statements>
  WHEN <value> then
    <statements>
  ...
  ELSE
    <statements>
END CASE;
```

```
CASE
  WHEN <condition> then
    <statements>
  WHEN <condition> then
    <statements>
  ...
  ELSE
    <statements>
END CASE;
```

SQL/PSM (cont'd.)

- Constructs for looping

```
[label]: REPEAT  
    <statement list>  
UNTIL <condition>  
END REPEAT [label];
```

```
[begin_label:] WHILE <condition> DO  
    <statement list>  
END WHILE [end_label]
```

```
FOR <loop name> AS <cursor name> CURSOR FOR <query> DO  
    <statement list>  
END FOR;
```

Repeat Until Example

```
DELIMITER $$ ;

CREATE FUNCTION CalcIncome ( starting_value INT )
RETURNS INT
NO SQL
BEGIN
    DECLARE income INT;
    SET income = 0;
    label1: REPEAT
        SET income = income + starting_value;
        UNTIL income >= 4000
    END REPEAT label1;
    RETURN income;
END; $$

DELIMITER ; $$
```

```
mysql> select CalcIncome(1000);
+-----+
| CalcIncome(1000) |
+-----+
|                4000 |
+-----+
1 row in set (0.00 sec)
```

- DELIMITER \$\$
- NO SQL

Loop Flow Control

- Iterate <label> – start the loop again
 - Can only be issued within LOOP, REPEAT, or WHILE statements
 - Works much like the “continue” statement in Java or C++.
- Leave <label> – jumps out of the control construct that has the given label.
 - Can only be issued within LOOP, REPEAT, or WHILE statements, just like the iterate statement.
 - You can use this at any level of nesting, → you can jump out to the out of the outermost loop if you desire.

Example:

write a procedure to update "TOTALSALARY" in the deptsal table

```
mysql> select * from employee;
```

fname	minit	lname	ssn	bdate	address	sex	salary	super_ssn	dno
JOHN	B	SMITH	123456789	2022-10-07	731 FONDREN, HOUSTON, TX	M	30000	333445555	5
FRANKLIN	T	WONG	333445555	2022-10-07	638 VOSS,HOUSTON TX	M	40000	888665555	5
ALICIA	J	ZELAYA	999887777	2022-10-07	3321 CASTLE, SPRING, TX	F	25000	987654321	4
JENNIFER	S	WALLACE	987654321	2022-10-07	291 BERRY, BELLAIRE, TX	F	43000	888665555	4
RAMESH	K	NARAYAN	666884444	2022-10-07	975 FIRE OAK, HUMBLE, TX	M	38000	333445555	5
JOYCE	A	ENGLISH	453453453	2022-10-07	5631 RICE, HOUSTON, TX	F	25000	333445555	5
AHMAD	V	JABBAR	987987987	2022-10-07	980 DALLAS, HOUSTON, TX	M	25000	987654321	4
JAMES	E	BORG	888665555	2022-10-07	450 STONE, HOUSTON, TX	M	55000		1

8 rows in set (0.00 sec)

```
mysql> select * from department;
```

dnumber	dname	mgr_ssn	mgr_start_date
5	RESEARCH	333445555	2022-10-07
4	ADMINISTRATION	987654321	2022-10-07
1	HEADQUARTERS	888665555	2022-10-07

3 rows in set (0.00 sec)

```
mysql> create table deptsal as
```

```
  -> select dnumber, 0 as totalsalary from department;
```

Query OK, 3 rows affected (0.01 sec)

Records: 3 Duplicates: 0 Warnings: 0

```
mysql> select * from deptsal;
```

dnumber	totalsalary
5	0
4	0
1	0

3 rows in set (0.00 sec)

Simple and Efficient Version

- DELIMITER \$\$
- CREATE PROCEDURE UpdateDeptSalary()
- BEGIN
- UPDATE deptsal d
- SET totalsalary = (
■ SELECT IFNULL(SUM(e.salary), 0)
■ FROM employee e
■ WHERE e.dno = d.dnumber
-);
- END\$\$
- DELIMITER ;

Alternative Version

- DELIMITER \$\$

- CREATE PROCEDURE UpdateDeptSalary()
- BEGIN
- UPDATE deptsal d
- LEFT JOIN (
■ SELECT dno, SUM(salary) AS total
■ FROM employee
■ GROUP BY dno
■) e ON e.dno = d.dnumber
- SET d.totalsalary = IFNULL(e.total, 0);
- END\$\$

- DELIMITER ;

Example:

write a function that returns increased salary by amount

```
mysql> select fname, lname, salary, giveRaise(salary, 0.1) as newsal  
-> from employee;
```

fname	lname	salary	newsal
JOHN	SMITH	30000	33000
FRANKLIN	WONG	40000	44000
ALICIA	ZELAYA	25000	27500.00000000000004
JENNIFER	WALLACE	43000	47300.000000000001
RAMESH	NARAYAN	38000	41800
JOYCE	ENGLISH	25000	27500.00000000000004
AHMAD	JABBAR	25000	27500.00000000000004
JAMES	BORG	55000	60500.000000000001

8 rows in set (0.00 sec)

```
DELIMITER $$  
CREATE FUNCTION giveRaise(current_salary DECIMAL(10,2),  
                           raise_percent DECIMAL(5,2) )  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
    RETURN current_salary * (1 + raise_percent);  
END$$  
DELIMITER ;
```


Is it 4:15 yet?

Modify so that procedure updates only department number passed as argument

```
mysql> call updateSalary(1);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> call updateSalary(4);  
Query OK, 1 row affected (0.00 sec)
```

```
mysql> call updateSalary(5);  
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select * from deptsal;
```

dnumber	totalsalary
5	133000
4	93000
1	55000

```
3 rows in set (0.00 sec)
```

■ Change

- Create procedure updateSalary (IN param1 int)
- d.dnumber to param1

- So the procedure must:
 - Accept a parameter (department name).
 - IN dept_name VARCHAR(20)
 - Find the department number from the department table.
 - JOIN department dep ON dep.dnumber = d.dnumber
 - Update only that department in deptsal.
 - WHERE dep.dname = dept_name

```
DELIMITER $$
```

```
CREATE PROCEDURE UpdateDeptSalary(IN dept_name VARCHAR(20))  
BEGIN  
    UPDATE deptsal d  
    LEFT JOIN (  
        SELECT dno, SUM(salary) AS total  
        FROM employee  
        GROUP BY dno  
    ) e ON e.dno = d.dnumber  
    JOIN department dep ON dep.dnumber = d.dnumber  
    SET d.totalsalary = IFNULL(e.total,0)  
    WHERE dep.dname = dept_name;  
END$$
```

```
DELIMITER ;
```

Break (4:30 PM – 4:50 PM)

Please be punctual so that
we restart on time

CHAPTER 14

Basics of Functional Dependencies and Normalization for Relational Databases

Chapter Outline

- 1 Informal Design Guidelines for Relational Databases
 - 1.1 Semantics of the Relation Attributes
 - 1.2 Redundant Information in Tuples and Update Anomalies
 - 1.3 Null Values in Tuples
 - 1.4 Spurious Tuples
- 2 Functional Dependencies (FDs)
 - 2.1 Definition of Functional Dependency
- 3 Normal Forms Based on Primary Keys
 - 3.1 Normalization of Relations
 - 3.2 Practical Use of Normal Forms
 - 3.3 Definitions of Keys and Attributes Participating in Keys
 - 3.4 First Normal Form
 - 3.5 Second Normal Form
 - 3.6 Third Normal Form
- 4 General Normal Form Definitions for 2NF and 3NF (For Multiple Candidate Keys)
- 5 BCNF (Boyce-Codd Normal Form)
- 6 Multivalued Dependency and Fourth Normal Form
- 7 Join Dependencies and Fifth Normal Form

1. Informal Design Guidelines for Relational Databases (1)

- What is relational database design?
 - The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
 - The logical "user view" level
 - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
 - - 1NF (First Normal Form)
 - - 2NF (Second Normal Form)
 - - 3NF (Third Normal Form)
 - - BCNF (Boyce-Codd Normal Form)
- Additional types of dependencies, further normal forms, relational design algorithms by synthesis are discussed in Chapter 15

1.1 Semantics of the Relational Attributes must be clear

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
 - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
 - Only foreign keys should be used to refer to other entities
 - Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

A simplified COMPANY relational database schema

EMPLOYEE

F.K.

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------

P.K.

DEPARTMENT

F.K.

Dname	<u>Dnumber</u>	Dmgr_ssn
-------	----------------	----------

P.K.

DEPT_LOCATIONS

F.K.

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

P.K.

PROJECT

F.K.

Pname	<u>Pnumber</u>	Plocation	Dnum
-------	----------------	-----------	------

P.K.

WORKS_ON

F.K.

F.K.

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

P.K.

Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

Fname	Minit	Lname	<u>Ssn</u>	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jenniter	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

Dname	<u>Dnumber</u>	Mgr_ssn	Mgr_start_date
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

<u>Essn</u>	<u>Pno</u>	Hours
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

Pname	<u>Pnumber</u>	Plocation	Dnum
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	Sex	Bdate	Relationship
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1988-01-04	Son
123456789	Alice	F	1986-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

1.2 Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
 - Wastes storage
 - Causes problems with update anomalies
 - Insertion anomalies
 - Deletion anomalies
 - Modification anomalies

EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Update Anomaly:
 - Changing the name of project number P1 from “Billing” to “Customer-Accounting” causes this update to be made for all 100 employees working on project P1.

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
 - Cannot insert a project unless an employee is assigned to it.
- Conversely
 - Cannot insert an employee unless assigned to a project.

EXAMPLE OF A DELETE ANOMALY

- Consider the relation:
 - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
 - When a project is deleted, it will result in deleting all the employees who work on that project.
 - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

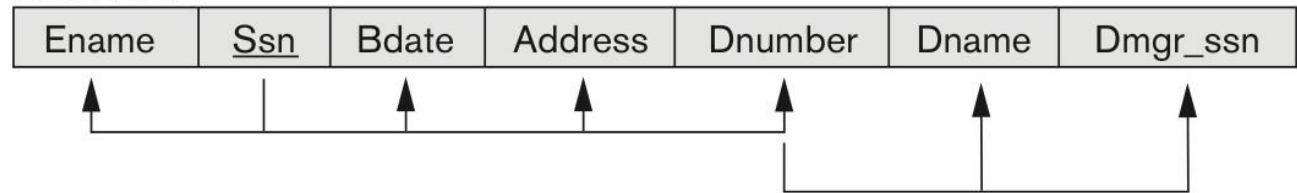
Figure 14.3 Two relation schemas suffering from update anomalies

Figure 14.3

Two relation schemas suffering from update anomalies. (a) EMP_DEPT and (b) EMP_PROJ.

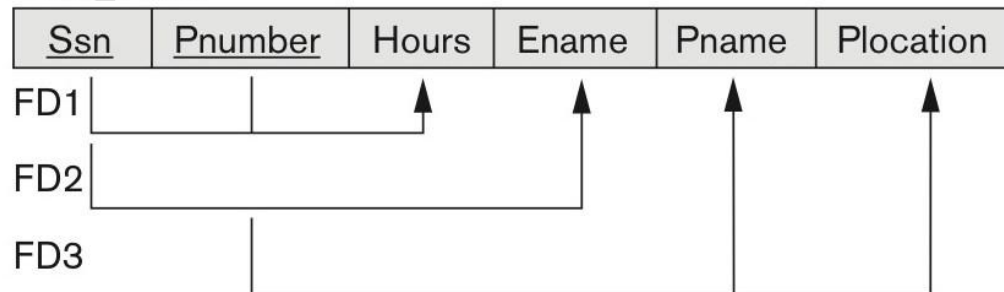
(a)

EMP_DEPT



(b)

EMP_PROJ



Sample states for EMP_DEPT and EMP_PROJ

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Guideline for Redundant Information in Tuples and Update Anomalies

■ GUIDELINE 2:

- Design a schema that does not suffer from the insertion, deletion and update anomalies.
- If there are any anomalies present, then note them so that applications can be made to take them into account.

1.3 Null Values in Tuples

- **GUIDELINE 3:**
 - Relations should be designed such that their tuples will have as few NULL values as possible
 - Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- **Reasons for nulls:**
 - Attribute not applicable or invalid
 - Attribute value unknown (may exist)
 - Value known to exist, but unavailable

1.4 Generation of Spurious Tuples – avoid at any cost

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations
- **GUIDELINE 4:**
 - The relations should be designed to satisfy the lossless join condition.
 - No spurious tuples should be generated by doing a natural-join of any relations.

Spurious Tuples (2)

- There are two important properties of decompositions:
 - a) Non-additive or losslessness of the corresponding join
 - b) Preservation of the functional dependencies.
- Note that:
 - Property (a) is extremely important and cannot be sacrificed.
 - Property (b) is less stringent and may be sacrificed. (See Chapter 15).

2. Functional Dependencies

- Functional dependencies (FDs)
 - Are used to specify *formal measures* of the "goodness" of relational designs
 - And keys are used to define **normal forms** for relations
 - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

2.1 Defining Functional Dependencies

- $X \rightarrow Y$ holds if whenever two tuples have the same value for X , they *must have* the same value for Y
 - For any two tuples $t1$ and $t2$ in any relation instance $r(R)$: If $t1[X]=t2[X]$, *then* $t1[Y]=t2[Y]$
- $X \rightarrow Y$ in R specifies a *constraint* on all relation instances $r(R)$
- Written as $X \rightarrow Y$; can be displayed graphically on a relation schema (denoted by an arrow).
- FDs are derived from the real-world constraints on the attributes

Examples of FD constraints (1)

- Social security number determines employee name
 - $SSN \rightarrow ENAME$
- Project number determines project name and location
 - $PNUMBER \rightarrow \{PNAME, PLOCATION\}$
- Employee ssn and project number determines the hours per week that the employee works on the project
 - $\{SSN, PNUMBER\} \rightarrow HOURS$

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance $r(R)$
- If K is a key of R , then K functionally determines all attributes in R
 - we never have two distinct tuples with $t1[K]=t2[K]$

Defining FDs from instances

- Note that in order to define the FDs, we need to understand the meaning of the attributes involved and the relationship between them.
- An FD is a property of the attributes in the schema R
- Given the instance (population) of a relation, all we can conclude is that an FD may exist between certain attributes.
- What we can definitely conclude is – that certain FDs do not exist because there are tuples that show a violation of those dependencies.

Figure 14.7 Ruling Out FDs

Note that given the state of the TEACH relation, we can say that the FD: Text $\rightarrow$ Course may exist. However, the FDs Teacher $\rightarrow$ Course, Teacher $\rightarrow$ Text and Course $\rightarrow$ Text are ruled out.

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

Figure 14.8 What FDs may exist?

- A relation $R(A, B, C, D)$ with its extension.
- Which FDs may exist in this relation?

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

3.1 Normalization of Relations (1)

■ Normalization:

- The process of decomposing unsatisfactory "bad" relations
 - by breaking up their attributes into smaller relations
- Needed:
 - Minimize data redundancy to
 - Avoid anomalies
 - Conserve storage space
 - Simplify enforcement of referential integrity constraints
 - Simplify maintaining (insert, update, delete) data.
 - Improve real world representation for a stronger basis for future growth.

■ Normal form:

- Highest normal form condition that the relation meets
- Indicates the degree to which the relation has been normalized

Normalization of Relations (2)

- 2NF, 3NF, BCNF
 - based on keys and FDs of a relation schema
- 4NF
 - based on keys, multi-valued dependencies : MVDs;
- 5NF
 - based on keys, join dependencies : JDs
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation; see Chapter 15)

3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
 - (usually up to 3NF and BCNF. 4NF rarely used in practice.)
- **Denormalization:**
 - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

3.3 Definitions of Keys and Attributes Participating in Keys (1)

- **superkey** of a relation schema $R = \{A_1, A_2, \dots, A_n\}$ is a
 - set of attributes S *subset-of* R with the property that
 - no two tuples t_1 and t_2 in any legal relation state r of R
 - will have $t_1[S] = t_2[S]$
- **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey anymore.

Definitions of Keys and Attributes Participating in Keys (2)

- If a relation schema has more than one key, each is called a **candidate** key.
 - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

3.4 First Normal Form

- Disallows
 - composite attributes
 - multivalued attributes
 - **nested relations**; attributes whose values for an *individual tuple* are non-atomic
- Considered to be part of the definition of a relation
- Most RDBMSs allow only those relations to be defined that are in First Normal Form

Normalization into 1NF

(a)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
-------	----------------	----------	------------

(b)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

1NF Normalizing of nested relations

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, Alicia J.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

<u>Ssn</u>	Ename
------------	-------

EMP_PROJ2

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------

3.5 Second Normal Form (1)

- Uses the concepts of **FDs, primary key**
- Definitions
 - **Prime attribute:** An attribute that is member of the primary key K
 - **Full functional dependency:** a FD $Y \rightarrow Z$ where removal of any attribute from Y means the FD does not hold any more
- Examples:
 - $\{SSN, PNUMBER\} \rightarrow HOURS$ is a full FD since
 - neither $SSN \rightarrow HOURS$ nor $PNUMBER \rightarrow HOURS$ hold
 - $\{SSN, PNUMBER\} \rightarrow ENAME$ is not a full FD
 - (it is called a partial dependency)
 - since $SSN \rightarrow ENAME$ also holds

Second Normal Form (2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization or “second normalization”

Normalizing into 2NF and 3NF

(a)

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

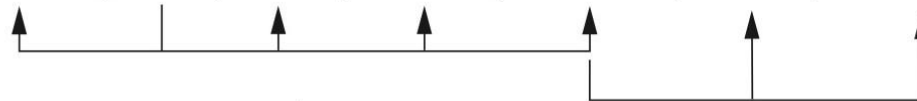
<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



(b)

EMP_DEPT

Ename	<u>Ssn</u>	Bdate	Address	Dnumber	Dname	Dmgr_ssn
-------	------------	-------	---------	---------	-------	----------



3NF Normalization

ED1

Ename	<u>Ssn</u>	Bdate	Address	Dnumber
-------	------------	-------	---------	---------



ED2

<u>Dnumber</u>	Dname	Dmgr_ssn
----------------	-------	----------



3.6 Third Normal Form (1)

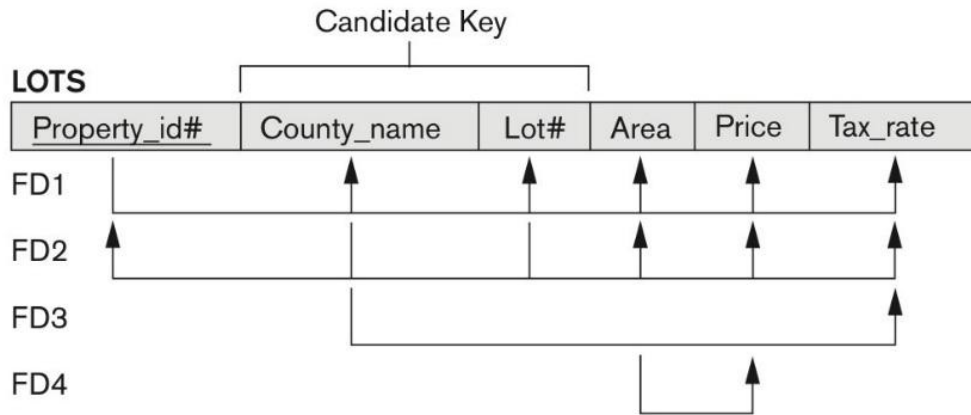
- Definition:
 - **Transitive functional dependency:**
 - a FD $X \rightarrow Z$ that
 - can be derived from two FDs $X \rightarrow Y$ and $Y \rightarrow Z$
- Examples:
 - $SSN \rightarrow DMGRSSN$ is a **transitive** FD Since
 - $SSN \rightarrow DNUMBER$ and
 - $DNUMBER \rightarrow DMGRSSN$ hold
 - $SSN \rightarrow ENAME$ is **non-transitive**
 - Since there is no set of attributes X where
 - $SSN \rightarrow X$ and $X \rightarrow ENAME$

Third Normal Form (2)

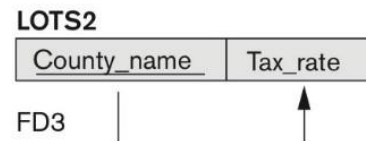
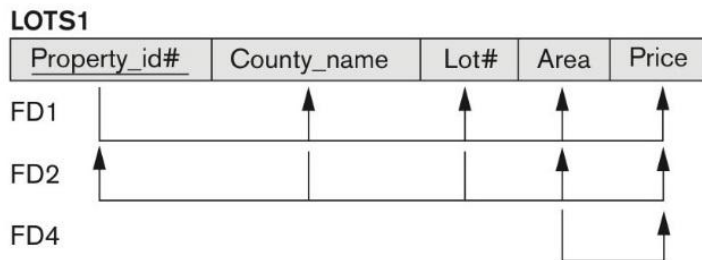
- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
 - In $X \rightarrow Y$ and $Y \rightarrow Z$, with X as the primary key, we consider this a problem only if Y is not a candidate key.
 - When Y is a candidate key, there is no problem with the transitive dependency .
 - E.g., Consider EMP (SSN, Emp#, Salary).
 - Here, $SSN \rightarrow Emp\# \rightarrow Salary$ and $Emp\#$ is a candidate key.

Normalization into 2NF and 3NF

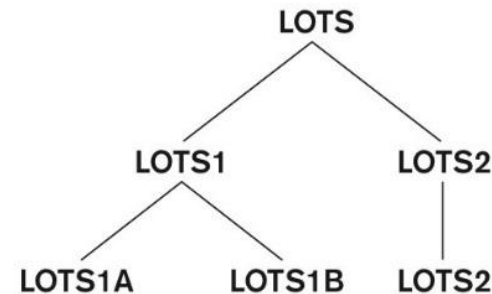
(a)



(b)



(d)

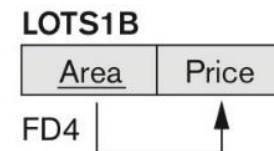
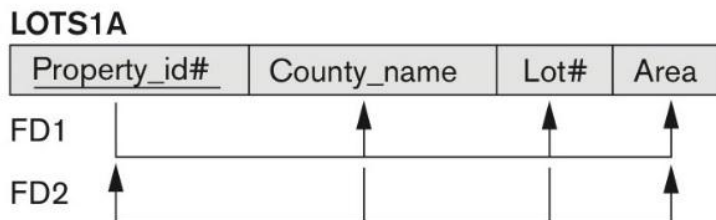


1NF

2NF

3NF

(c)



Normal Forms Defined Informally

- 1st normal form
 - All attributes depend on **the key**
- 2nd normal form
 - All attributes depend on **the whole key**
- 3rd normal form
 - All attributes depend on **nothing but the key**

4. General Normal Form Definitions (For Multiple Keys) (1)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- Any attribute involved in a candidate key is a prime attribute
- All other attributes are called non-prime attributes.

4.1 General Definition of 2NF (For Multiple Candidate Keys)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every* key of R
- In Figure 14.12 the FD $\text{County_name} \rightarrow \text{Tax_rate}$ violates 2NF.

So second normalization converts LOTS into
LOTS1 (Property_id#, County_name, Lot#, Area, Price)
LOTS2 (County_name, Tax_rate)

4.2 General Definition of Third Normal Form

- Definition:
 - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
 - A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R, then either:
 - (a) X is a superkey of R, or
 - (b) A is a prime attribute of R
- LOTS1 relation violates 3NF because
 $\text{Area} \rightarrow \text{Price}$; and Area is not a superkey in LOTS1.

4.3 Interpreting the General Definition of Third Normal Form

- Consider the 2 conditions in the Definition of 3NF:
A relation schema R is in **third normal form (3NF)** if whenever a FD $X \rightarrow A$ holds in R , then either:
 - (a) X is a superkey of R , or
 - (b) A is a prime attribute of R
- Condition (a) catches two types of violations :
 - one where a prime attribute functionally determines a non-prime attribute. This catches 2NF violations due to non-full functional dependencies.
 - two where a non-prime attribute functionally determines a non-prime attribute. This catches 3NF violations due to a transitive dependency.

4.3 Interpreting the General Definition of Third Normal Form (2)

- **ALTERNATIVE DEFINITION of 3NF:** We can restate the definition as:

A relation schema R is in **third normal form (3NF)** if every non-prime attribute in R meets both of these conditions:

- It is fully functionally dependent on every key of R
- It is non-transitively dependent on every key of R

Note that stated this way, a relation in 3NF also meets the requirements for 2NF.

- The condition (b) from the last slide takes care of the dependencies that “slip through” (are allowable to) 3NF but are “caught by” BCNF which we discuss next.

5. BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD** $X \rightarrow A$ holds in R , then X is a **superkey** of R
- Each normal form is strictly stronger than the previous one
 - Every 2NF relation is in 1NF
 - Every 3NF relation is in 2NF
 - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- Hence BCNF is considered a **stronger form of 3NF**
- The goal is to have each relation in BCNF (or 3NF)

Figure 14.13 Boyce-Codd normal form

(a)

LOTS1A

<u>Property_id#</u>	County_name	Lot#	Area
---------------------	-------------	------	------



BCNF Normalization

LOTS1AX

<u>Property_id#</u>	Area	Lot#
---------------------	------	------

LOTS1AY

<u>Area</u>	County_name
-------------	-------------

(b)

R

<u>A</u>	<u>B</u>	C
----------	----------	---



Figure 14.14 A relation TEACH that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 14.14
A relation TEACH that is in 3NF
but not BCNF.

Achieving the BCNF by Decomposition (1)

- Two FDs exist in the relation TEACH:
 - fd1: { student, course} -> instructor
 - fd2: instructor -> course
- {student, course} is a candidate key for this relation
 - So this relation is in 3NF *but not in BCNF*
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
 - D1: {student, instructor} and {student, course}
 - D2: {course, instructor} and {course, student}
 - D3: {instructor, course} and {instructor, student} ✓
- All three decompositions will lose fd1.
 - We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).
- A test to determine whether a binary decomposition (decomposition into two relations) is non-additive (lossless) is discussed under Property NJB on the next slide. We then show how the third decomposition above meets the property.

Test for checking non-additivity of Binary Relational Decompositions

- **Testing Binary Decompositions for Lossless Join (Non-additive Join) Property**
 - **Binary Decomposition:** Decomposition of a relation R into two relations.
 - **PROPERTY NJB (non-additive join test for binary decompositions):**
 - A decomposition $D = \{R_1, R_2\}$ of R has the lossless join property with respect to a set of functional dependencies F on R *if and only if* either
 - The f.d. $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+ , or
 - The f.d. $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+ .
- **NJB means:**
 - The natural Join of the decomposed relations produces the original relation without generating extra tuples

Test for checking non-additivity of Binary Relational Decompositions

If you apply the NJB test to the 3 decompositions of the TEACH relation:

- D1 gives
 - **Student** → Instructor or **Student** → Course,
 - none of which is true.
- D2 gives
 - **Course** → Instructor or **Course** → Student,
 - none of which is true.
- in D3 we get
 - **Instructor** → Course or **Instructor** → Student.
 - **Instructor** → Course is indeed true,
 - the NJB property is satisfied and D3 is determined as a non-additive (good) decomposition.

Example

Original relation:

EmpID	DeptID	DeptName
1	5	Research
2	4	Admin

Join:

SQL

```
SELECT *  
FROM employee e  
JOIN department d  
ON e.deptid = d.deptid;
```

Result:

EmpID	DeptID	DeptName
1	5	Research
2	4	Admin

Decomposition:

EMPLOYEE

EmpID	DeptID
1	5
2	4

DEPARTMENT

DeptID	DeptName
5	Research
4	Admin

Example

Original:

Student	Course	Instructor
Alice	DB	Smith

If another student exists:

Student	Course
Bob	DB

Joining produces:

Student	Course	Instructor
Alice	DB	Smith
Bob	DB	Smith

The tuple **Bob-DB-Smith** may be incorrect → **spurious tuple**.

This decomposition **does not satisfy NJD**.

Decompose into:

R1(Student, Course)

Student	Course
Alice	DB

R2(Course, Instructor)

Course	Instructor
DB	Smith

Chapter Summary

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
- Normal Forms (1NF, 2NF, 3NF) Based on Primary Keys
- General Normal Form Definitions of 2NF and 3NF (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)
- Fourth and Fifth Normal Forms
 - optional reading in hidden slides

Is it 6:05 yet?

Prove only 3rd decomposition is BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

- Three possible decompositions for relation TEACH
 - D1:
 - {student, instructor} and
 - {student, course}
 - D2:
 - {course, instructor} and
 - {course, student}
 - D3:
 - {instructor, course} and
 - {instructor, student} ✓

Review Deliverables (6:15 – 6:20)

■ Homework 08:

- Due By: Wednesday March 18th, 2026, 11:59 PM
- Available Until: Friday March 20th, 2026, 11:59 PM

■ Lab 06:

- Due By: Wednesday March 18th, 2026, 11:59 PM
- Available Until: Friday March 20th, 2026, 11:59 PM

■ Project Phase 04:

- Due By: Friday April 3rd, 2026, 11:59 PM
- Available Until: Sunday April 5th, 2026 11:59 PM